

The Database Revolution

Topic 2

The Database Revolution

The database revolution represents a fundamental transformation in how data is stored, managed, and accessed in computer systems. Prior to modern database systems, **data management was tightly coupled** with application programs, making systems difficult to maintain, scale, and secure.

The Database Revolution

The emergence of **database management systems (DBMS)** and, more importantly, **relational database models** reshaped computing and became a cornerstone of **modern information systems** and **web applications**.

From Flat Files to Structured Data Management

Before the widespread adoption of database systems, most organizations relied on **flat file systems** to store data.

Flat files are **simple text** or **binary files** where data is stored **sequentially**, often with **fixed-length records**.

Each application maintained its **own data files**, resulting in **data redundancy, inconsistency**, and **tight coupling** between programs and data structures.

In flat file systems, **any change in data format required modifications to all programs** that accessed the file.

Which means ...

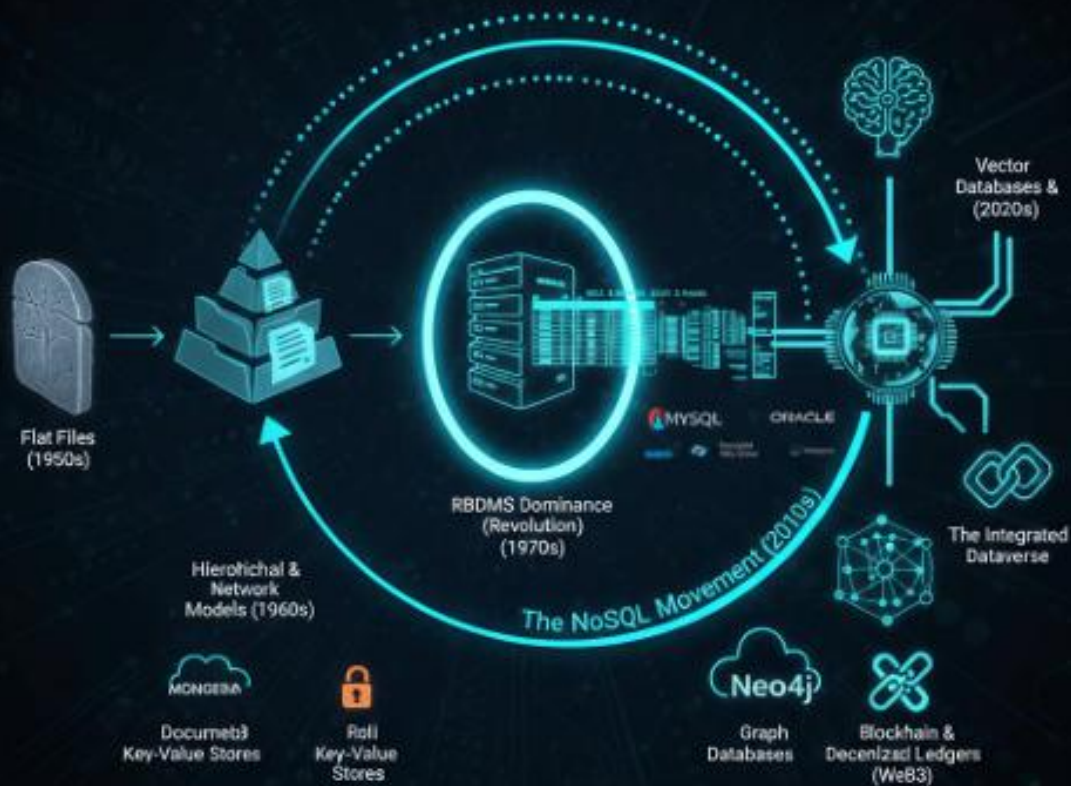
Data sharing across applications was **difficult**, and **enforcing data integrity rules** was largely the responsibility of the programmer.

However,

As data volumes grew and organizations required more reliable information systems, the **limitations of flat file storage became increasingly evident.**

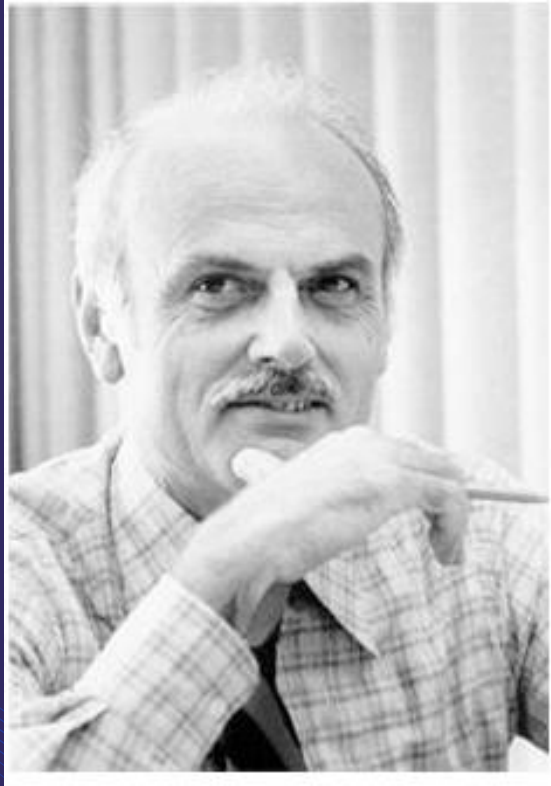
This growing complexity created the need for a **centralized, structured approach** to data management, paving the way for **database systems.**

The Database Revolution: From Stone to AI-Driven Insights



Next

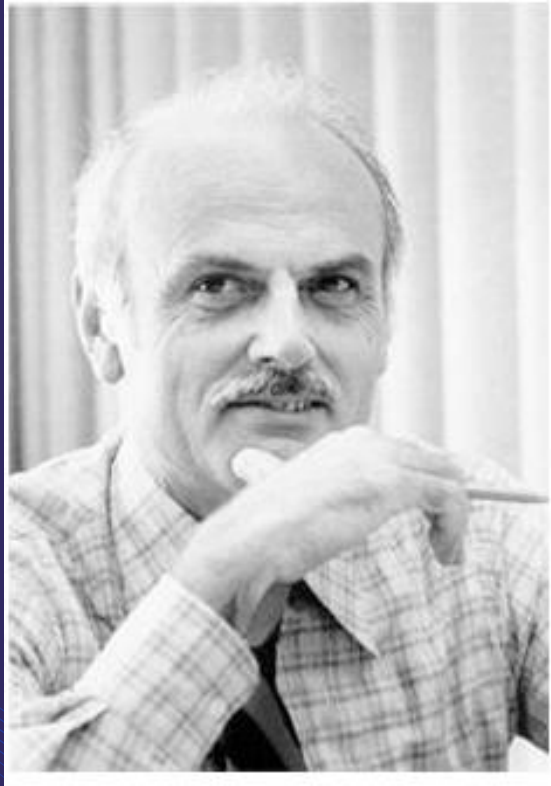
The Emergence of the Relational Model



The most significant breakthrough in database technology came in **1970** when **Dr. Edgar F. Codd**, a researcher at **IBM**, introduced the relational database model

Edgar F. Codd

International Business Machines Corporation



Codd proposed representing data as relations, which are essentially tables consisting of **rows (tuples)** and **columns (attributes)**.

This abstraction separated the **logical structure** of data from its **physical storage**, allowing systems to evolve without disrupting applications.

Edgar F. Codd

International Business Machines Corporation

The relational model emphasized **data independence**, **mathematical foundations**, and **formal query mechanisms**. Instead of navigating through files and pointers, users could declare **what data** they wanted rather than **how** to retrieve it. This concept revolutionized data access and laid the foundation for modern database systems.

Which means ...

Codd's 12 Rules

These rules define the principles for maintaining **data integrity, consistency, and accessibility** in relational systems. While **no** commercial database fully implements all 12 rules, they serve as a guiding framework for relational database design.

The Information Rule

All data should be represented logically as **values in tables (relations)**, where each row corresponds to a unique record and each column represents a specific attribute. This ensures a uniform and consistent representation of data.

Rule
01

Guaranteed Access Rule

Every individual data item (atomic value) must be accessible using a **combination of table name, primary key, and column name**, ensuring that all data is reachable without ambiguity.

Rule
02

Systematic Treatment of Null Values

Null values, representing **missing** or **inapplicable data**, must be handled systematically. The database must distinguish between **unknown**, **missing**, and **inapplicable data**, allowing for proper query results and calculations.

Rule
03

Dynamic Online Catalog Based on the Relational Model

The database schema, metadata, and definitions should themselves be stored in relational tables. This allows users to query and manipulate the database structure using the **same relational operations** used for normal data.

Rule
04

Comprehensive Data Sub-language Rule

The system must support at least **one relational language** that allows data definition, data manipulation, and data control, providing a consistent way to manage all aspects of the database.

Rule
05

View Updating Rule

The system should allow updates through **views**, which are **virtual tables** derived from **base tables**. Any changes to a view should propagate correctly to the underlying data whenever possible.

Rule
06

High-level Insert, Update, and Delete

The system must support **set-at-a-time operations** instead of requiring **record-by record processing**. This allows **bulk insertion, deletion, and modification** of data in a **single command**.

Rule
07

Physical Data Independence

Applications should remain **unaffected** by **changes in physical storage structures**. Moving data to different files or reorganizing storage should not require modification of application programs.

Rule
08

Logical Data Independence

Applications should remain unaffected by changes in the **logical structure** of the database, such as **adding or removing columns or tables**, provided the core data remains intact.

Rule
09

Integrity Independence

Integrity constraints, such as **uniqueness**, **referential integrity**, and **domain rules**, must be stored in the database **not in the application programs**. This ensures centralized enforcement of **data integrity**.

Rule
10

Distribution Independence

The database should function correctly regardless of **distribution across multiple locations**. Users should not need to know whether data resides on a single server or multiple distributed nodes.

Rule
11

Non-subversion Rule

If a relational system provides **lower-level access methods**, these should **not bypass the integrity rules** and **constraints** enforced at the relational level. This prevents **corruption or inconsistent data access**.

Rule
12

The Rise of SQL and the 1990s Database Boom



The relational model gained widespread adoption with the development of **Structured Query Language (SQL)**.

SQL provided a standardized, declarative way to define, manipulate, and query relational data

During the **1980s and especially the 1990s**, **SQL-based RDBMS** such as **Oracle, IBM DB2, Microsoft SQL Server, and MySQL** became dominant in enterprise and academic environments.

The Rise of SQL and the 1990s Database Boom

The 1990s marked a turning point as databases moved from **specialized systems** to **mainstream business infrastructure**. Client-server computing, graphical user interfaces, and the rise of the World Wide Web accelerated the need for robust database backends.

SQL databases became the preferred choice for **handling transactions, user data, and content management**.

The Rise of SQL and the 1990s Database Boom

Compared to flat file systems, SQL-based databases offered data integrity, concurrency control, security, and recovery mechanisms, making them suitable for mission-critical applications.

The standardization of SQL further enabled portability and interoperability across platforms.

Impact on Web and Database Integration

The database revolution directly enabled the development of **dynamic web applications**. **Server-side scripting languages and frameworks** relied on **relational databases** to store and retrieve data efficiently.

Features such as **user authentication**, **online transactions**, **content management systems**, and **analytics** became possible due to the structured and reliable nature of relational databases.

Impact on Web and Database Integration

Even with the emergence of NoSQL and non-relational systems in the 21st century, relational databases remain foundational in web development, largely due to their strong theoretical basis and widespread SQL support.

The database revolution transformed data management from **isolated flat files** into **centralized, structured, and reliable relational systems**. Codd's relational model and 12 rules provided the theoretical foundation for modern databases, while SQL became the dominant language for data interaction in the 1990s. This shift enabled scalable, secure, and data-driven applications and remains central to web and database integration today.

In summary



DID YOU KNOW?

- Edgar F. Codd initially faced resistance at **IBM** because his ideas challenged existing database products.
- SQL was originally called **SEQUEL** (**Structured English Query Language**).
- Despite SQL's dominance, many early database systems in the 1990s only partially followed **Codd's 12 rules**.

Thanks!

Do you have any question?

CREDITS: This presentation template was created by Slidesgo, including icons by **Flaticon**, infographics & images by **Freepik**

